

Visão Computacional

Visão Computacional: Fundamentos, Técnicas e Aplicações no Contexto da Engenharia de Software

Disciplina: Inteligência Artificial (N3) **Professor:** Claudinei Dias (Ney) **Tema:** Visão Computacional **Trabalho:** Seminários — Aplicações Inteligentes no Dia a Dia

1. Introdução

A Visão Computacional é a subárea da Inteligência Artificial responsável por permitir que sistemas computacionais interpretem, processem e compreendam informações visuais — imagens estáticas ou vídeos — de forma análoga à percepção humana. Nas últimas duas décadas, o avanço combinado do poder computacional (especialmente GPUs), da disponibilidade de grandes bases de dados rotuladas e da evolução das redes neurais profundas transformou a Visão Computacional em uma das áreas de IA com maior impacto prático, presente em smartphones, veículos autônomos, sistemas de segurança, diagnósticos médicos e linhas de produção industriais.

Do ponto de vista da Engenharia de Software, a Visão Computacional deixou de ser um tema estritamente acadêmico para se tornar um componente de arquitetura de sistemas: modelos de reconhecimento de imagem são hoje empacotados como serviços (APIs de visão), bibliotecas (OpenCV, MediaPipe) e modelos treináveis (TensorFlow, PyTorch), exigindo do engenheiro de software conhecimento tanto do funcionamento interno dos algoritmos quanto das boas práticas de integração, versionamento de modelos e escalabilidade.

Este relatório tem como objetivo apresentar os fundamentos teóricos da Visão Computacional, as principais técnicas e ferramentas utilizadas atualmente, exemplos reais de aplicação, e propor uma implementação prática de um classificador de imagens construído com uma Rede Neural Convolucional (CNN) desenvolvida do zero.

2. Fundamentos Teóricos

2.1 O que é uma imagem para um computador

Para um computador, uma imagem digital é uma matriz de números. Uma imagem em escala de cinza é representada por uma matriz bidimensional (altura $\times$ largura), em que cada posição contém um valor de intensidade luminosa (normalmente de 0 a 255). Uma imagem colorida é representada por três dessas matrizes sobrepostas, uma para cada canal de cor (vermelho, verde e azul — RGB). Todo o processamento de Visão Computacional parte dessa representação numérica.

2.2 Processamento clássico de imagens

Antes da popularização do aprendizado profundo, a Visão Computacional dependia fortemente de técnicas de processamento de sinais e extração manual de características (*feature engineering*), como:

- **Filtros de convolução** (Sobel, Gaussiano, Laplaciano) para detecção de bordas e suavização de ruído;
- **Segmentação** (limiarização/threshold, watershed) para separar objetos do fundo;
- **Descritores de características** como SIFT, SURF e HOG, que extraem pontos-chave e padrões geométricos de uma imagem, usados posteriormente em algoritmos clássicos de classificação (SVM, k-NN).

Essas técnicas ainda são amplamente usadas em pipelines de pré-processamento, mesmo em soluções modernas baseadas em redes neurais.

2.3 Redes Neurais Convolucionais (CNNs)

O marco que popularizou a Visão Computacional moderna foi a Rede Neural Convolucional (*Convolutional Neural Network* — CNN), uma arquitetura de rede neural profunda especializada em dados com estrutura de grade, como imagens. Uma CNN é composta tipicamente por:

- **Camadas convolucionais:** aplicam filtros (kernels) que deslizam sobre a imagem, aprendendo automaticamente a detectar padrões como bordas, texturas e, em camadas mais profundas, formas e objetos complexos;
- **Funções de ativação** (como ReLU): introduzem não linearidade, permitindo que a rede aprenda relações complexas;
- **Camadas de pooling** (max pooling, average pooling): reduzem a dimensionalidade espacial, tornando a rede mais eficiente e robusta a pequenas variações de posição do objeto na imagem;
- **Camadas totalmente conectadas (Dense):** ao final da rede, combinam as características extraídas para realizar a classificação final;
- **Função de perda e retropropagação (backpropagation):** o erro entre a predição e o rótulo real é calculado e propagado para trás na rede, ajustando os pesos por meio de um otimizador (como Adam ou SGD).

A grande vantagem das CNNs em relação às técnicas clássicas é que elas aprendem automaticamente quais características extrair da imagem, eliminando a necessidade de engenharia manual de atributos.

2.4 Tarefas fundamentais em Visão Computacional

- **Classificação de imagens:** atribuir um rótulo a uma imagem inteira (ex.: “gato” ou “cachorro”);
- **Detecção de objetos:** localizar e classificar múltiplos objetos em uma imagem, geralmente delimitando-os com caixas (*bounding boxes*);
- **Segmentação semântica/instância:** classificar cada pixel da imagem, distinguindo inclusive múltiplas instâncias de um mesmo objeto;
- **Reconhecimento facial e biometria:** identificar ou verificar a identidade de uma pessoa a partir de características faciais.

3. Técnicas, Ferramentas e Frameworks

Categoria	Ferramentas/Frameworks	Uso típico
Processamento clássico	OpenCV	Filtros, detecção de bordas, manipulação de imagem em tempo real
Deep Learning	TensorFlow/Keras, PyTorch	Construção e treinamento de CNNs
Modelos pré-treinados	ResNet, VGG, MobileNet, EfficientNet	Transfer learning para classificação
Detecção de objetos	YOLO (You Only Look Once), Faster R-CNN, SSD	Detecção em tempo real
Rostos e pontos-chave corporais	MediaPipe, Dlib, FaceNet	Biometria, rastreamento corporal
Ambiente de desenvolvimento	Google Colab, Jupyter Notebook	Treinamento com GPU gratuita, prototipagem
Implantação (deploy)	TensorFlow Lite, ONNX, TensorRT	Execução em dispositivos móveis/embarcados

Um ponto importante do ponto de vista de Engenharia de Software é a diferença entre **treinar** um modelo (processo caro computacionalmente, geralmente feito uma vez ou periodicamente) e **servir** um modelo (executar inferências em produção, que precisa ser rápido, escalável e muitas vezes rodar em dispositivos com recursos limitados). Isso motiva técnicas como quantização e poda de modelos, além do uso de formatos otimizados como TensorFlow Lite para aplicações móveis.

4. Exemplos Reais e Estudos de Caso

Biometria em smartphones: sistemas como o reconhecimento facial (Face ID) e leitura de impressão digital utilizam redes neurais convolucionais especializadas para extrair características únicas do rosto ou digital do usuário e compará-las com um modelo armazenado localmente no dispositivo, sem enviar os dados biométricos brutos para a nuvem — uma decisão de arquitetura motivada tanto por desempenho quanto por privacidade.

Veículos autônomos: empresas como Tesla e Waymo utilizam múltiplas câmeras combinadas com redes de detecção de objetos em tempo real para identificar pedestres, veículos, faixas de pista e sinalizações, integrando essas informações a sistemas de tomada de decisão que controlam direção, aceleração e frenagem.

Diagnóstico médico por imagem: modelos de CNN treinados sobre grandes volumes de exames (radiografias, tomografias, imagens de retina) auxiliam médicos na detecção precoce de doenças como câncer de pele, retinopatia diabética e certos tipos de tumores, muitas vezes atingindo acurácia comparável à de especialistas humanos em tarefas específicas.

Controle de qualidade industrial: linhas de produção utilizam câmeras e modelos de visão para inspecionar produtos em tempo real, identificando defeitos de fabricação com velocidade e consistência superiores à inspeção manual.

Agricultura de precisão: drones equipados com câmeras e modelos de visão computacional identificam pragas, deficiências nutricionais em plantações e estimam a produtividade de safras, otimizando o uso de insumos agrícolas.

5. Proposta de Aplicação Prática: Classificador de Imagens com CNN do Zero

Como demonstração prática, propõe-se a construção de uma Rede Neural Convolucional **do zero** (sem uso de transfer learning), treinada para a tarefa de classificação de imagens utilizando o dataset **CIFAR-10**, composto por 60.000 imagens coloridas de 32x32 pixels, divididas em 10 classes (avião, automóvel, pássaro, gato, cervo, cachorro, sapo, cavalo, navio e caminhão).

A implementação será disponibilizada em um notebook Google Colab / repositório GitHub, contendo:

1. Carregamento e normalização do dataset;
2. Definição da arquitetura da CNN (camadas convolucionais, pooling, dropout e camadas densas);
3. Treinamento do modelo com acompanhamento de métricas de acurácia e perda (treino e validação);
4. Avaliação do modelo em um conjunto de teste, com matriz de confusão;
5. Interface simples para realizar previsões em novas imagens.

5.1 Arquitetura proposta

```
Entrada (32x32x3)
↓
Conv2D (32 filtros, 3x3) + ReLU
MaxPooling2D (2x2)
↓
Conv2D (64 filtros, 3x3) + ReLU
MaxPooling2D (2x2)
↓
Conv2D (64 filtros, 3x3) + ReLU
↓
Flatten
Dense (64) + ReLU
Dropout (0.5)
Dense (10) + Softmax
```

5.2 Código de referência (Python / Keras)

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import cifar10
```

```

# 1. Carregamento e normalização dos dados
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0

# 2. Definição da arquitetura da CNN
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])

# 3. Compilação e treinamento
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(x_train, y_train, epochs=15,
                    validation_data=(x_test, y_test))

# 4. Avaliação
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"Acurácia no conjunto de teste: {test_acc:.2%}")

```

5.3 Resultados esperados

Com essa arquitetura relativamente simples, espera-se uma acurácia entre 70% e 80% no conjunto de teste do CIFAR-10 após aproximadamente 15 épocas de treinamento — um resultado didático que evidencia tanto o potencial quanto as limitações de uma CNN construída sem técnicas mais avançadas (como data augmentation, batch normalization ou arquiteturas pré-treinadas), abrindo espaço para discussão sobre possíveis melhorias durante a apresentação.

5.4 Entrega

O código completo, o notebook de treinamento e as instruções de execução serão disponibilizados em um repositório público no GitHub (ou notebook Google Colab), permitindo que qualquer pessoa reproduza o experimento.

6. Conclusão

A Visão Computacional evoluiu de técnicas baseadas em processamento clássico de sinais e extração manual de características para arquiteturas de aprendizado profundo capazes de aprender representações complexas diretamente a partir dos dados. Essa evolução ampliou drasticamente o leque de aplicações práticas, impactando setores como saúde, mobilidade, indústria, segurança e agricultura.

Do ponto de vista da Engenharia de Software, trabalhar com Visão Computacional exige não apenas conhecimento de algoritmos de aprendizado de máquina, mas também competências de engenharia: escolha de frameworks adequados, otimização de modelos para inferência em produção, e integração desses modelos em sistemas maiores de forma escalável e confiável.

A implementação prática de um classificador CNN do zero, ainda que simples, ilustra de forma concreta os conceitos teóricos apresentados neste relatório e serve como base para explorações mais avançadas, como transfer learning, detecção de objetos e segmentação de imagens.

7. Referências

- GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. MIT Press, 2016.
- SZELISKI, R. *Computer Vision: Algorithms and Applications*. 2ª ed. Springer, 2022.
- Documentação oficial do TensorFlow: <https://www.tensorflow.org/>
- Documentação oficial do OpenCV: <https://opencv.org/>

- Documentação oficial do PyTorch: <https://pytorch.org/>
- REDMON, J. et al. *You Only Look Once: Unified, Real-Time Object Detection*. CVPR, 2016.
- KRIZHEVSKY, A. *Learning Multiple Layers of Features from Tiny Images* (dataset CIFAR-10), 2009.